SecuritizeBridgeV2 Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Stalin

Alix40

March 30, 2026

Contents

1 About Cyfrin **2**

2 Disclaimer **2**

3 Risk Classification **2**

4 Protocol Summary **2**

5 Audit Scope **2**

6 Executive Summary **2**

7 Findings **4**

7.1 Medium Risk 4

7.1.1 `USDCBridgeV2::_quoteBridge` hardcodes `msgValue=0` creating fee mismatch that bricks the USDC bridge when gas dropoff is configured 4

7.1.2 In-transit same-wallet VAA overwrites compliance attribute changes made on destination chain via unconditional `Registry::updateInvestor` in `SecuritizeBridge::executeVAAv1` . 5

7.2 Low Risk 7

7.2.1 Permanent loss of DSTokens when bridging to non-EVM chains via the backward-compatible `SecuritizeBridge::bridgeDSTokens` due to missing target chain type validation 7

7.3 Gas Optimization 8

7.3.1 `SecuritizeBridge::_bridgeDSTokensInternal` calls `publishMessage` before fee sufficiency check, wasting gas on revert 8

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

SecuritizeBridge enables compliant cross-chain transfers to EVM and non-EVM chains of regulated DS tokens using the Wormhole Executor Framework. Investor KYC/compliance attributes are validated on source and synchronized to destination during bridging.

5 Audit Scope

The audit scope was limited to:

```
https://github.com/securitize-io/bc-securitize-bridge-sc/tree/feature/BC-1823 at commitHash
↪  d5765fdc72136a2566e33f19af4663188655428f
- contracts/bridge/SecuritizeBridge.sol
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the SecuritizeBridgeV2 smart contracts provided by Securitize. The findings consist of 2 Medium & 1 Low severity issues with the remainder being gas optimizations.

Of the two medium severity findings:

- 7.1.1 results in a temporary DoS of the primary function used to bridge USDC across chains
- 7.1.2 allows an attacker to permanently nullify compliance revocations across all chains. `SecuritizeBridge::executeVAAv1` unconditionally overwrites the destination chain's compliance state with the VAA's KYC snapshot, applying no check against the current attributes beforehand. An investor facing revocation can frontrun it with a same-wallet bridge, capturing a pre-revocation snapshot; replaying the VAA then causes `Registry::updateInvestor` to silently restore their compliant status.

Operational Notes

SecuritizeBridges will be deployed with new proxies

The existing deployed Bridge contracts will be paused and their roles will be revoked from the rest of the system. New proxies will be deployed for the new version of the SecuritizeBridge contracts integrating with the Wormhole Executor framework with support to non-evm chains.

Summary

Project Name	SecuritizeBridgeV2
Repository	bc-securitize-bridge-sc
Commit	d5765fdc7213...
Fix Commit	328f890f5726...
Audit Timeline	March 17th - March 20th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	2
Low Risk	1
Informational	0
Gas Optimizations	1
Total Issues	4

Summary of Findings

[M-1] `USDCBridgeV2::_quoteBridge` hardcodes `msgValue=0` creating fee mismatch that bricks the USDC bridge when gas dropoff is configured	Resolved
[M-2] In-transit same-wallet VAA overwrites compliance attribute changes made on destination chain via unconditional `Registry::updateInvestor` in `SecuritizeBridge::executeVAAv1`	Acknowledged
[L-1] Permanent loss of DSTokens when bridging to non-EVM chains via the backward-compatible `SecuritizeBridge::bridgeDSTokens` due to missing target chain type validation	Resolved
[G-1] `SecuritizeBridge::_bridgeDSTokensInternal` calls `publishMessage` before fee sufficiency check, wasting gas on revert	Acknowledged

7 Findings

7.1 Medium Risk

7.1.1 `USDCBridgeV2::_quoteBridge` hardcodes `msgValue=0` creating fee mismatch that bricks the USDC bridge when gas dropoff is configured

Description: `USDCBridgeV2::_quoteBridge` computes the executor fee using `RelayInstructions.encodeGas(gasLimit, 0)` with a hardcoded zero for the gas dropoff parameter:

```
function _quoteBridge(uint16 _targetChain) private view returns (uint256 execFee) {
    bytes memory request = ExecutorMessages.makeCCTPv2Request();
@>  bytes memory relayInstructions = RelayInstructions.encodeGas(gasLimit, 0); // hardcoded 0
    execFee = executorQuoterRouter.quoteExecution(
        _targetChain, bytes32(0), address(this), quoterAddr, request, relayInstructions
    );
}
```

However, `USDCBridgeV2::sendUSDCCrossChainDeposit` calls `requestExecution` with the actual stored `msgValue`:

```
executorQuoterRouter.requestExecution{value: execFee}(
    _targetChain,
    bytes32(0),
    address(this),
    quoterAddr,
    ExecutorMessages.makeCCTPv2Request(),
@>  RelayInstructions.encodeGas(gasLimit, msgValue) // actual msgValue
);
```

When admin sets `msgValue > 0` via `USDCBridgeV2::updateMsgValue`, the fee quoted by `_quoteBridge` is lower than what the executor actually requires for the relay instructions passed to `requestExecution`. The balance check at L215 (`address(this).balance < execFee`) uses this underestimated fee, allowing the transaction to proceed with insufficient ETH for the actual execution cost.

Impact: When admin configures `msgValue > 0` via `USDCBridgeV2::updateMsgValue`, the USDC bridge is completely bricked — every `USDCBridgeV2::sendUSDCCrossChainDeposit` call reverts and no USDC can be bridged until admin resets `msgValue` to 0.

The Wormhole `ExecutorQuoterRouter::requestExecution` (source) re-computes the required fee from the **actual relay instructions passed in the same call** (not from a prior quote). It parses the `relayInstructions` bytes to extract `gasLimit` and `msgValue`, converts them to source chain value, and checks `msg.value >= requiredPayment`. If insufficient, it reverts with `Underpaid(provided, expected)`.

Since the revert occurs within the same atomic transaction, the preceding USDC `safeTransferFrom` and CCTP `depositForBurn` are also rolled back — no USDC is permanently lost. However, the bridge is non-functional for its intended purpose: bridging USDC with gas dropoff configured.

Recommended Mitigation: Use `msgValue` in `_quoteBridge` to match the relay instructions actually passed to `requestExecution`:

```
function _quoteBridge(uint16 _targetChain) private view returns (uint256 execFee) {
    bytes memory request = ExecutorMessages.makeCCTPv2Request();
-   bytes memory relayInstructions = RelayInstructions.encodeGas(gasLimit, 0);
+   bytes memory relayInstructions = RelayInstructions.encodeGas(gasLimit, msgValue);
    execFee = executorQuoterRouter.quoteExecution(
        _targetChain, bytes32(0), address(this), quoterAddr, request, relayInstructions
    );
}
```

Securitize: Fixed in commit c313304

Cyfrin: Verified. Call to `RelayInstructions::encodeGas` is now called with `msgValue` instead of hardcoding the value to0

7.1.2 In-transit same-wallet VAA overwrites compliance attribute changes made on destination chain via unconditional `Registry::updateInvestor` in `SecuritizeBridge::executeVAAv1`

Description: `SecuritizeBridge::executeVAAv1` processes incoming cross-chain messages on the destination chain. For same-wallet transfers (`sourceWallet == destinationAddress`), it unconditionally calls `Registry::updateInvestor` with the investor's compliance attributes — country, KYC status, accreditation, qualification, and professional flags, along with their expiration timestamps — decoded from the VAA payload:

```
// SecuritizeBridge::executeVAAv1
...
  if (sourceWallet == destinationAddress) {
      ...
      registryService.updateInvestor(investorId, investorId, country, investorWallets, attributeIds,
      ↪  attributeValues, attributeExpirations);
  }
...
```

These attributes are a snapshot captured on the source chain at the moment the investor initiated the bridge. Between that moment and when the Wormhole executor delivers the VAA to the destination chain, the investor's compliance state on the destination chain may have been independently updated by the Securitize compliance team — a routine operation that happens whenever KYC is renewed, an attribute expires, a sanction is applied, or investor status changes. There is no check on the destination chain against the current compliance state before overwriting it. When the VAA lands, `Registry::updateInvestor` fires and silently restores the in-transit snapshot, discarding any changes that were made to the destination registry while the message was in flight.

The unintentional case requires no deliberate action from the investor. A normal bridge transaction running concurrently with a compliance update is sufficient: the investor initiates a same-wallet bridge on the source chain, the compliance team updates the investor's attributes on the destination chain for any legitimate reason, and the executor delivers the VAA — causing `Registry::updateInvestor` to overwrite the compliance team's changes with the now-stale bridge payload. The destination registry is left in the state it was in at bridge initiation time, not in the state the compliance team intended.

The same mechanism also enables deliberate exploitation. An investor who becomes aware that a KYC revocation or sanctions flag is imminent can frontrun it by initiating a same-wallet bridge on the source chain moments before the revocation is applied. The resulting VAA carries a snapshot of their pre-revocation attributes. Once the revocation lands on the destination chain, the investor submits the VAA — or simply waits for the executor to deliver it — and `Registry::updateInvestor` restores their compliant status, nullifying the revocation. Because `SecuritizeBridge::executeVAAv1` is a public function with no caller restriction, the investor can time the submission themselves if the executor has not yet delivered the VAA. Wormhole VAAs do not expire, so the submission window is unbounded.

In a regulated securities context, both cases represent a meaningful compliance failure. A KYC revocation, a sanctions flag, or an attribute expiration applied to the destination chain is silently rolled back when a VAA executes, regardless of whether the overwrite was accidental or intentional. The investor retains or regains the compliance status they should no longer hold.

Impact: The delivery window for the unintentional race condition spans from when the source-chain VAA is signed by the Wormhole guardian until the executor submits it on the destination chain. At `consistencyLevel` 200 (instant), this window is on the order of seconds to minutes. At `consistencyLevel` 201 (safe, ~13 minutes) or higher finality thresholds, the window is wider. Any compliance state change on the destination chain that falls within this window is vulnerable to being overwritten.

For the intentional frontrunning case, the window is effectively unlimited. The investor initiates the bridge before the revocation, and the resulting VAA remains valid indefinitely. Even if the executor delivers it promptly, a brief delay between bridge initiation and VAA execution is sufficient if the revocation has not yet propagated to the destination chain at the time of delivery.

An additional edge case further extends the attack surface: any revert of `SecuritizeBridge::executeVAAv1` leaves the VAA unconsumed and permanently resubmittable. The `whenNotPaused` modifier on the Bridge is applied at the `SecuritizeBridge::executeVAAv1` function entry point, meaning that if the destination bridge is paused at the time of delivery, the executor's delivery transaction reverts before `isVaaConsumed[vm.hash]` is ever set to `true`.

The VAA is left unconsumed. The investor can re-use the VAA at any point in the future to restore its privileges to the snapshot

Proof of Concept:

1. Investor has full KYC on Chain A (Accredited=1, Qualified=1, Professional=1). They bridge tokens to Chain B (same-wallet) via `bridgeDSTokens` — tokens burn on A, a VAA is published with `targetChain = Chain B` carrying Chain A's elevated KYC snapshot. On Chain B, the investor is flagged `liquidate only` (or the bridge is paused for maintenance). The executor tries to deliver — `executeVAAv1` reverts, `isVaaConsumed` rolls back. The VAA stays unconsumed on the Wormhole guardian network.
2. Time passes. Admin revokes Accredited/Qualified/Professional on **both** Chain A and Chain B for legitimate compliance reasons. The liquidate-only flag is also lifted (or bridge unpaused) since it was a temporary measure.
3. The investor replays the unconsumed VAA on Chain B (its target chain) by calling `executeVAAv1`. The same-wallet branch fires: `updateInvestor` at L365 writes the stale elevated KYC (Accredited=1, Qualified=1, Professional=1) over Chain B's revoked state. Since `updateInvestor` runs before `issueTokens`, `preIssuanceCheck` reads the just-restored elevated attributes from the registry and passes — tokens are minted. Chain B's KYC is now restored to pre-revocation values.
4. The investor immediately bridges from Chain B $\rightarrow$ Chain A (same-wallet) via `bridgeDSTokens`. `_validateLockedTokens` does not check KYC status — only token lock periods — so the bridge proceeds. `_getInvestorData` reads Chain B's just-restored elevated KYC and encodes it into a fresh VAA with `targetChain = Chain A`. This VAA is delivered on Chain A — `updateInvestor` overwrites Chain A's revoked KYC with the elevated attributes from Chain B.
5. Both chains now have the investor's revoked compliance attributes silently restored. A single stale VAA cascaded across chains, undoing the admin's revocation everywhere.

Recommended Mitigation: To prevent investors from bypassing a state where they have been removed/restricted from interacting with the platform. Add a new bool var, i.e. `bridgingBlocked`, and check :

- if it's true, revert (governance should set that var to true when updating the investor's attributes to a state in which they should no longer be able to use the DSToken)
- If it's false, then allow execution - The var being false accounts for the case when the investor is not registered on that chain. and. when the investor is still authorized

Additionally, track the state off-chain and override any wrong updates.

Securitize: Acknowledged. Governance is responsible for properly restricting investors (e.g., fully locked or removed); subsequent operations (including issuance/transfers) would fail at other enforcement layers.

7.2 Low Risk

7.2.1 Permanent loss of DSTokens when bridging to non-EVM chains via the backward-compatible `SecuritizeBridge::bridgeDSTokens` due to missing target chain type validation

Description: `SecuritizeBridge.bridgeDSTokens(uint16 _targetChain, uint256 _value)` is the backward-compatible bridge entry point for users who want to bridge DS tokens to the same wallet address on a destination chain. To derive the destination address, it encodes `msg.sender` (a 20-byte EVM address) into a 32-byte value using EVM-specific padding:

```
// SecuritizeBridge.bridgeDSTokens(uint16 _targetChain, uint256 _value)
bytes32 destinationAddress = bytes32(uint256(uint160(_msgSender())));
```

This encoding is only valid for EVM-compatible chains (Ethereum, Arbitrum, Avalanche, Base, Optimism, Polygon), where addresses are 20 bytes padded to 32. Non-EVM chains supported by the bridge — specifically Solana (Wormhole chain ID 1) — use a different address scheme: Ed25519 public keys, which are natively 32 random bytes and bear no structural relationship to EVM addresses.

If a user calls the backward-compatible `SecuritizeBridge::bridgeDSTokens` and it passes the `_targetChain` as the ID of a non-EVM chain, the burnt bridged tokens are permanently unrecoverable.

The precondition is a user calling `SecuritizeBridge::bridgeDSTokens` — the simpler, backward-compatible entry point — with a non-EVM target chain ID. Unlike `SecuritizeBridge.bridgeDSTokensToAddress(uint16 _targetChain, uint256 _value, bytes32 _destinationAddress)`, which requires callers to supply a chain-appropriate bytes32 destination and is therefore more explicitly a power-user interface.

The call path is:

1. `bridgeDSTokens(_targetChain=1, _value)` — encodes `msg.sender` as `bytes32(uint256(uint160(msg.sender)))` (EVM padding)
2. Calls `bridgeDSTokensToAddress(1, _value, malformedBytes32)` — no zero-check or chain-type check
3. `_bridgeDSTokensInternal()` — validates investor compliance, burns tokens, publishes VAA with malformed destination
4. Executor delivers VAA to Solana bridge
5. Solana bridge issues tokens to the 32-byte value — an address the user does not own

No validation between steps 1 and 2 checks whether `_targetChain` corresponds to an EVM chain before applying EVM-specific address encoding.

Recommended Mitigation: Add a check in `SecuritizeBridge::bridgeDSTokens` that reverts if `_targetChain` is not an EVM-compatible chain before deriving the destination address from `msg.sender`. Maintain an allowlist of EVM Wormhole chain IDs (or a mapping of chain ID to address type), and revert with a descriptive error when a non-EVM chain ID is supplied to the backward-compatible entry point.

Securitize: Fixed in commit 328f890.

Cyfrin: Verified. Added a check in `SecuritizeBridge::bridgeDSTokens` to revert if `_targetChain` is not an EVM-compatible chain.

7.3 Gas Optimization

7.3.1 `SecuritizeBridge::_bridgeDSTokensInternal` calls `publishMessage` before fee sufficiency check, wasting gas on revert

Description: In `SecuritizeBridge::_bridgeDSTokensInternal`, `IWormhole::publishMessage` at L440 and `IExecutorQuoterRouter::quoteExecution` at L451 are both called before the fee sufficiency check at L460:

```
// L440 - publishes message, costs significant gas
uint64 sequence = _wormholeCore.publishMessage{value: coreFee}(0, payload, consistencyLevel);

// L451-458 - quotes execution
uint256 execFee = _executorQuoterRouter.quoteExecution(...);

// L460 - fee check happens AFTER expensive operations
if (msg.value < coreFee + execFee) revert InsufficientETHForFees();
```

If the fee check reverts, EVM atomicity rolls back everything, so no funds are lost. However, significant gas is wasted executing `publishMessage` and `quoteExecution` before determining that the user sent insufficient ETH.

The contract already has `SecuritizeBridge::_quoteBridge` (L584-602) which demonstrates how to pre-compute fees using `IWormhole::nextSequence` instead of the actual published sequence, making it feasible to check fees first.

Recommended Mitigation: Move the fee check before `publishMessage` by using `nextSequence` to pre-compute the quote:

```
uint256 coreFee = _wormholeCore.messageFee();
uint64 sequence = _wormholeCore.nextSequence(address(this));
bytes memory request = ExecutorMessages.makeVAAv1Request(_whChainId, _addressToBytes32(address(this)),
↪  sequence);
bytes memory relayInstructions = RelayInstructions.encodeGas(gasLimit, msgValue);
uint256 execFee = _executorQuoterRouter.quoteExecution(
    _targetChain, targetBridgeAddress, _msgSender(), _quoterAddr, request, relayInstructions
);
if (msg.value < coreFee + execFee) revert InsufficientETHForFees();

// Now proceed with publishMessage and burn
```

Securitize: Acknowledged.